Dokumentasi Ekosistem Aplikasi Internal PT Kertas Padalarang

Rancangan awal · Oktober 2026 · Disusun dari analisis lima aplikasi lama dan dokumen rancangan ekosistem

1. Ringkasan

PT Kertas Padalarang (PTKP), anak usaha Peruri, memiliki lima aplikasi internal yang dibangun terpisah dan tidak saling terhubung: PTKP SIM NEO, INSYSPRO HRIS, E-Arsip, SI ELANG, dan SI WOPPIT. Kelimanya akan dibangun ulang dari nol sebagai satu ekosistem terintegrasi.

Pembangunan dilakukan bertahap oleh satu pengembang. Karena itu langkah pertama bukan membangun fitur, melainkan membangun pondasi integrasi: identitas pengguna, struktur organisasi, hak akses, dan layanan bersama. Setelah pondasi siap, aplikasi dibangun satu per satu di atasnya, dimulai dari E-Arsip.

Dokumen ini menjelaskan apa yang akan dibangun, mengapa, bagaimana arsitekturnya, serta bagian mana yang sudah diputuskan dan mana yang masih terbuka.

2. Kondisi saat ini

2.1 Lima aplikasi yang ada

Aplikasi	Fungsi	Teknologi	Penyimpanan sandi
PTKP SIM NEO	Sistem informasi manufaktur: produksi kertas sekuriti (Mesin Kertas, QC lab, Finishing, Sortier, Pakzaal, Ekspedisi), sistem manajemen terpadu dan CAPA (tiket PTKP/TKK), pemeliharaan mesin (CMMS), audit internal, akuntansi, absensi WFH	PHP, MySQL	MD5 tanpa salt

Aplikasi	Fungsi	Teknologi	Penyimpanan sandi
INSYSPRO HRIS	Kepegawaian, absensi fingerprint (ZKTeco), penggajian, PPh 21, BPJS, rekrutmen, medical	WinForms (desktop 2-tier), PostgreSQL	Plaintext
E-Arsip	Persuratan, penomoran surat otomatis, arsip digital, nota dinas dan memo	CodeIgniter 3, MySQL	Bcrypt (Ion Auth)
SI ELANG	E-learning: pre-test, materi PDF, post-test, evaluasi kesan, sertifikat	CodeIgniter, MySQL (55 tabel)	MD5 tanpa salt
SI WOPPIT	Work order dan tiket perbaikan IT (nomor ITxxx)	PHP, MySQL	Bcrypt

2.2 Skala

Perkiraan pengguna sekitar 300 karyawan dengan beban server sangat rendah. Data lama berjumlah ratusan ribu baris, dengan pertumbuhan puluhan sampai ratusan ribu baris per tahun. Log fingerprint adalah yang tercepat tumbuh, sekitar 440 ribu baris per tahun untuk 300 karyawan. Skala ini sangat kecil untuk PostgreSQL; performa dijaga oleh indeks dan query yang baik, bukan oleh jumlah database.

3. Masalah

3.1 Banyak bug di semua aplikasi

Karena kode terpisah-pisah, setiap perbaikan harus dilakukan berkali-kali di tempat berbeda.

3.2 Tidak ada integrasi

- Setiap aplikasi punya tabel pengguna sendiri, sehingga satu orang memiliki lima akun dan lima sandi.
- Sandi disimpan dengan tiga cara berbeda: MD5 tanpa salt, plaintext, dan bcrypt.
- Struktur organisasi (departemen, unit kerja, kelas) tersimpan lima kali dan hampir pasti sudah tidak sinkron. Bentuknya pun berbeda-beda: HRIS memakai SBU, divisi, departemen, bagian; SIM memakai departemen dan work center; E-Arsip memakai

departemen dan unit kerja; SI ELANG memakai pohon kelas (self-reference); SI WOPPIT memakai daftar departemen datar.

3.3 Fungsi yang tumpang tindih

- Absensi ada di SIM dan di HRIS.
- Work order ada di SI WOPPIT dan di CMMS pada SIM.
- Penomoran otomatis dibuat sendiri di tiap aplikasi.

3.4 Pola celah keamanan yang sama berulang

Pola	Contoh di sistem lama
Server mempercayai input dari browser	SIM menjalankan SQL mentah yang dikirim klien (base64); SI WOPPIT menerima `user_role_id` dan `id_user` dari field tersembunyi sehingga pengguna biasa bisa menjadi Administrator; SI ELANG menentukan alur belajar lewat `localStorage`
Otorisasi tanpa pemeriksaan kepemilikan data (IDOR)	E-Arsip pada `edit` dan `delete`; SI WOPPIT menampilkan tiket semua departemen ke semua pengguna
Penomoran dengan `MAX()+1`	E-Arsip dan SI WOPPIT dapat menghasilkan nomor ganda bila dua orang menyimpan bersamaan
Kredensial terbuka	SIM dan HRIS menyimpan kredensial database dalam teks biasa di berkas konfigurasi; HRIS memakai akun superuser database
Hash sandi usang	MD5 tanpa salt di SIM dan SI ELANG; sandi plaintext di HRIS
Berkas sensitif terbuka	`error.log` sekitar 246 MB di root web SI WOPPIT; unggahan berkas di direktori publik pada E-Arsip

3.5 Skema warisan dan redundansi data

- E-Arsip masih membawa kolom sisa sistem ujian online (`opsi_a` sampai `opsi_e`, `jawaban`, `bobot`) dan peran bernama `trainer` dan `peserta`.

- Kolom `sent_trainer_id` di E-Arsip berisi data dipisahkan koma dan dicari dengan `FIND_IN_SET()`, sehingga indeks tidak bisa dipakai.
- Nota dinas disimpan ganda (satu baris keluar, satu baris masuk) sehingga rawan tidak sinkron.
- SI ELANG memiliki empat modul penilaian yang hampir 95% identik, banyak tabel MyISAM tanpa foreign key.
- HRIS menyalin banyak kolom master pegawai ke tabel gaji; SIM menyimpan nama petugas berulang di tabel produksi.
- SIM memiliki banyak versi tabel yang tumpang tindih tanpa alat migrasi.

4. Tujuan

- Satu identitas per orang dan satu login untuk semua aplikasi.
- Satu sumber kebenaran untuk data pegawai dan struktur organisasi, dipakai semua aplikasi.
- Hak akses dikelola terpusat dan ditegakkan di server pada setiap aksi.
- Layanan bersama (penomoran, berkas, notifikasi, audit, persetujuan) dibangun sekali dan dipakai semua aplikasi.
- Pola bug lama tidak terulang karena dicegah oleh fondasi, bukan oleh kehati-hatian per fitur.
- Pembangunan bisa dilakukan bertahap tanpa mengganggu aplikasi lama yang masih berjalan.

Bukan tujuan saat ini

- Membangun fitur aplikasi secara mendalam. Itu dikerjakan per aplikasi setelah pondasi siap.
- Memecah sistem menjadi microservices.

5. Keputusan yang sudah diambil

Aspek	Keputusan
Backend	Laravel (PHP). Alasan: aplikasi kebanyakan CRUD, banyak fitur bawaan (autentikasi, antrean, notifikasi, migrasi, upload), dan sudah dikenal dari sistem lama

Aspek	Keputusan
Frontend	Vue, dihubungkan lewat Inertia sehingga tidak perlu membangun API terpisah
Database	Satu PostgreSQL, satu schema per modul
Bentuk aplikasi	Satu aplikasi Laravel dengan modul-modul terpisah (modular monolith)
Infrastruktur	Docker pada satu VM baru di server Proxmox milik kantor
Aplikasi pertama setelah pondasi	E-Arsip
Struktur organisasi	Struktur di HRIS (SBU, divisi, departemen, bagian) dikonfirmasi masih berlaku dan dipakai sebagai struktur resmi

Mengapa bukan microservices

Pengembang hanya satu orang, skala kecil (sekitar 300 karyawan), beban server sangat rendah, dan data saling terikat erat. Memecahnya menambah kompleksitas tanpa manfaat. Batas antar modul dibuat tegas sejak awal sehingga pemisahan di masa depan tetap murah bila diperlukan.

Mengapa satu aplikasi modular, bukan aplikasi terpisah yang dihubungkan lewat API

Keputusan: satu aplikasi Laravel yang dibagi menjadi modul-modul dengan batas tegas, dengan Platform sebagai inti. Pertimbangannya dinilai untuk jangka panjang (5 sampai 10 tahun), bukan untuk ukuran tim.

Aspek	Satu aplikasi modular	Aplikasi terpisah + API Platform
Alur lintas modul (mutasi pegawai, CAPA, payroll)	Satu transaksi atau satu event	Alur terdistribusi: bila satu langkah gagal, langkah lain harus dibatalkan atau diulang, dan itu harus dirancang, diuji, dan dipantau selamanya

Aspek	Satu aplikasi modular	Aplikasi terpisah + API Platform
Penomoran	Dipanggil dalam transaksi database yang sama dengan penyimpanan dokumen: tidak ada nomor ganda dan tidak ada nomor bolong	Beda database, sehingga nomor bisa bolong atau ganda bila penyimpanan gagal atau permintaan diulang; butuh mekanisme reservasi dan idempotency
Integritas data	`employee_id` dijaga foreign key database	Tidak ada foreign key lintas database; data yatim harus dijaga lewat kode
Pemeriksaan izin	Pemanggilan fungsi di memori	Panggilan API tiap aksi, atau cache izin yang harus diinvalidasi saat berubah
Bila Platform mati	Satu hal yang dijaga	Semua aplikasi tidak bisa login atau memeriksa izin
Perawatan dan upgrade	Satu framework, satu set dependensi, satu proses patch	Banyak repositori dan versi yang bisa melenceng; satu aplikasi tertinggal di versi lama yang tidak aman (persis kondisi lima aplikasi saat ini)
Tampilan dan portal pegawai	Satu shell dan satu desain	UI per aplikasi; portal gabungan butuh upaya tambahan
Login	Sesi Laravel biasa	Perlu SSO (OIDC/OAuth)

Kelebihan aplikasi terpisah yang nyata: isolasi kegagalan, irama rilis yang berbeda, teknologi yang berbeda, dan kecocokan dengan banyak tim. Karena itu pemisahan dipertimbangkan secara selektif (lihat bagian 14), bukan untuk semua modul.

Pemisahan nanti lebih murah daripada penyatuan nanti. Bila batas modul rapi dan Platform diakses lewat interface, modul mana pun bisa dikeluarkan menjadi aplikasi sendiri tanpa menulis ulang dari nol. Pemisahan layak dilakukan bila ada salah satu pemicu: ada tim atau pengembang lain, ada klien non-web (mobile, mesin fingerprint), satu modul membutuhkan jadwal rilis atau skala yang berbeda, atau gangguan di satu modul tidak boleh menyentuh modul lain.

Mengapa Inertia, bukan API terpisah

Dengan Inertia, controller Laravel langsung mengembalikan halaman Vue beserta datanya. Tidak ada lapisan REST atau JSON terpisah dan tidak ada token; login memakai sesi dan CSRF bawaan Laravel.

- **Kelebihan:** satu codebase; otorisasi tetap di satu tempat (controller dan policy), sesuai tujuan bahwa tidak ada keputusan hak akses yang bergantung pada browser; validasi, redirect, dan pesan galat sudah ditangani; tidak perlu JWT, refresh token, dan CORS.
- **Kekurangan:** frontend terikat ke Laravel dan tidak bisa dipakai klien lain; integrasi pihak luar tetap butuh endpoint HTTP; props halaman terlihat di sumber halaman sehingga controller hanya boleh mengirim field yang dibutuhkan (Resource atau DTO), tidak pernah model mentah.
- **Menjaga pintu tetap terbuka:** logika bisnis ditaruh di kelas layanan dan controller dibuat tipis. Bila API dibutuhkan (mobile, fingerprint, pihak ketiga), controller API dengan Sanctum cukup memanggil layanan yang sama.

6. Arsitektur

6.1 Gambaran umum

- Satu aplikasi Laravel berisi modul: Platform, Arsip, Layanan, Pembelajaran, SDM, Manufaktur dan Mutu.
- Satu database PostgreSQL berisi schema: `platform`, `arsip`, `layanan`, `elearning`, `sdm`, `mes`.
- Modul Platform adalah inti. Modul lain bergantung padanya, tidak sebaliknya.

6.2 Aturan batas modul

- Modul tidak boleh membaca tabel milik modul lain. Satu-satunya pengecualian adalah schema `platform` (pegawai, organisasi, hak akses).
- Tabel modul lain hanya menyimpan `employee_id` dan `org_unit_id`, tidak pernah menyalin nama pegawai atau nama unit.
- Komunikasi antar modul lewat kelas layanan yang jelas (misalnya Directory, Numbering, Files), atau lewat event untuk perubahan state.
- Setiap keputusan otorisasi dibuat di server.
- Setiap aplikasi dapat diberi akun database sendiri yang hanya boleh mengakses schema-nya.

6.3 Modul Platform (pondasi)

Pegawai dan organisasi

- `org_units`: pohon organisasi (SBU, divisi, departemen, bagian). Kolom `path` bertipe ltree dihitung otomatis sehingga pertanyaan seperti "semua unit di bawah divisi X" cukup satu query.
- `positions`: jabatan.
- `employees`: identitas inti pegawai (NIP sebagai kunci bisnis). Data HR sensitif seperti KTP, rekening, dan keluarga nanti berada di modul SDM, bukan di sini.
- `employee_assignments`: riwayat penempatan pegawai di unit dan jabatan, dengan tanggal mulai dan selesai. Mutasi berarti menutup satu baris dan membuka baris baru. Riwayat diperlukan agar payroll dan arsip tahu posisi pegawai pada waktu tertentu.

Akun dan hak akses

- `users`: satu orang satu akun, sandi di-hash dengan algoritma modern.
- `roles`, `permissions`, `role_permissions`, `user_roles`.
- Izin berbentuk `modul.sumber.aksi`, contoh `arsip.surat.create`.
- Setiap pemberian izin ke peran memiliki scope: `own` (milik sendiri), `unit` (unit sendiri), `unit_tree` (unit dan turunannya), `all` (semua).
- Penugasan peran ke pengguna dapat dibatasi pada unit tertentu dan memiliki masa berlaku.

Layanan bersama

- **Penomoran**: tabel format dan sekuens, dengan fungsi database atomik. Dipanggil di dalam transaksi yang sama dengan penyimpanan dokumen sehingga tidak ada nomor ganda dan tidak ada nomor bolong.
- **Berkas**: metadata di database, isi berkas di luar web root, unduhan lewat controller yang memeriksa izin, tipe berkas dideteksi dari isinya.
- **Audit log**: mencatat siapa, apa, kapan, nilai sebelum dan sesudah. Bersifat append-only dan dipartisi per tahun.
- **Pemetaan akun lama ke akun baru** (`legacy_identities`) untuk masa transisi.
- **Notifikasi** memakai fasilitas bawaan Laravel dan antrean. Saluran email dan WhatsApp ditambahkan saat dibutuhkan.

6.4 Pola otorisasi

- Setiap aksi diperiksa di server terhadap izin dan scope. Contoh: pengguna boleh mengubah surat X karena scope izinnya `unit` dan surat itu milik unitnya.

- Tidak ada keputusan hak akses yang bergantung pada field form, localStorage, atau input lain dari browser.
- Identitas pengguna pada setiap transaksi selalu diambil dari sesi server, bukan dari form.

6.5 Pemetaan dari model hak akses lama

Aplikasi lama	Model lama	Di model baru
SIM NEO	8 kelompok peran, 15 domain modul, tingkat izin R / R-W / R-W-E	Izin `mes.*` per sumber dan aksi; R-W-E dipecah menjadi aksi baca, ubah, hapus, dan override
HRIS	4 grup, ACL per menu (visible, insert, edit, delete)	Izin `sdm.*` per sumber dan aksi
E-Arsip	3 peran, tanda (O) untuk milik sendiri	Izin `arsip.*` dengan scope `own` atau `unit`
SI ELANG	admin, pengajar, siswa; pembatasan data milik sendiri	Izin `elearning.*` dengan scope `own`
SI WOPPIT	3 peran, ACL per halaman dan aksi	Izin `layanan.*`; pengguna biasa dibatasi `own`

6.6 Skala data dan jalur pertumbuhan

Bila di masa depan data membesar, urutan tindakannya: indeks dan pemeriksaan query lambat, partisi tabel berderet waktu, pengarsipan data lama, replika baca untuk laporan, dan terakhir memindahkan satu schema (misalnya `mes`) ke database sendiri.

6.7 Struktur modul di dalam aplikasi

Kode dikelompokkan per modul (domain), bukan per jenis berkas. Setiap modul memiliki service provider sendiri yang mendaftarkan route, migrasi, dan policy-nya.

```
app/
  Modules/
    Platform/   Models, Services (Directory, Numbering, Files, Audit),
Policies, Http, database/migrations, routes
    Arsip/      Models, Services, Policies, Http, database/migrations, routes
    Layanan/    WOPPIT (CMMS tetap di Manufaktur)
    Pembelajaran/   SI ELANG
    Sdm/        HRIS
    Manufaktur/ SIM NEO, dibagi submodul: Produksi, Mutu, Cmms, Akuntansi
resources/js/Modules/<Modul>/Pages/...
```

Alur satu permintaan: route modul, lalu controller tipis, lalu policy (izin dan scope dari Platform), lalu service modul (memanggil Numbering dan Files dari Platform), simpan ke schema modul, lalu `Inertia::render` ke halaman Vue.

Modul besar seperti SIM NEO dipecah menjadi submodul dengan aturan batas yang sama. Aturan batas dijaga otomatis oleh tes arsitektur (Deptrac atau Pest arch tests) yang gagal bila modul membaca tabel atau model modul lain. Frontend dipecah per halaman oleh Vite sehingga pengguna hanya mengunduh kode halaman yang dibuka. Folder modul dibuat saat gilirannya; mulai dengan `Platform` dan `Arsip`.

7. Modul yang akan dibangun

Urutan yang direncanakan. Setiap modul dibangun di atas Platform dan diuji ke dalam ekosistem sebelum modul berikutnya dimulai. Urutan ini adalah usulan awal dan boleh berubah.

1. **Platform (pondasi)**: identitas, organisasi, hak akses, penomoran, berkas, audit.
2. **E-Arsip**: modul nyata pertama. Menjadi pembuktian bahwa pondasi bekerja.
3. **Layanan** (SI WOPPIT): mesin tiket dan work order.
4. **Pembelajaran** (SI ELANG): kelas, materi, ujian, sertifikat.
5. **SDM** (HRIS): absensi, payroll, rekrutmen, medical. Paling sensitif, perlu masa paralel dengan sistem lama.
6. **Manufaktur dan Mutu** (SIM NEO): paling besar dan paling kritis bagi produksi, dikerjakan terakhir dan bertahap.

8. Contoh integrasi yang dituju

Integrasi lintas aplikasi inilah alasan ekosistem dibangun:

- **Pegawai baru atau mutasi** di modul SDM: akun, unit, dan hak akses terbentuk atau berubah otomatis, kelas pelatihan yang sesuai ditugaskan, dan surat keputusannya tercatat di E-Arsip.
- **Temuan CAPA/TKK yang butuh pelatihan**: modul Pembelajaran menugaskan kelas, lalu bukti kelulusan kembali ke modul Mutu sebagai bukti kompetensi untuk audit ISO.
- **Absensi, jadwal shift, dan payroll** berasal dari satu sumber data. Jadwal shift juga dapat dipakai sebagai sumber regu di produksi.
- **Satu portal pegawai**: slip gaji, absensi, cuti, pelatihan, tiket IT, dan surat. Ini sekaligus menutup kekurangan HRIS yang belum punya portal mandiri pegawai.
- **Penomoran, notifikasi, berkas, dan persetujuan berjenjang** memakai layanan yang sama di semua modul.

9. Infrastruktur

9.1 Kondisi server

Proxmox VE 8.2.2, satu node, 64 CPU dan sekitar 62 GiB RAM dengan rata-rata pemakaian sangat rendah. VM yang ada: 100 dan 101 (Windows), 103 (website), 104 (PTKP-SIM), 105 (PTKP-APPS-INTERNAL), 106 (RESTORE-TEST). Ada storage BACKUP-NAS dan VM RESTORE-TEST, artinya pengujian restore sudah menjadi kebiasaan.

9.2 Rencana penempatan

- Satu VM Linux baru (Debian atau Ubuntu LTS) untuk produksi, dan satu VM lebih kecil untuk pengembangan.
- Ukuran awal produksi sekitar 4 vCPU, 8 GB RAM, 80 GB disk. Dapat diubah.
- Docker Compose berisi: reverse proxy, aplikasi Laravel, worker antrean, PostgreSQL dengan volume terpisah.
- Jangan memakai LXC untuk Docker, dan jangan menumpang pada VM aplikasi lama.
- Backup dua lapis: snapshot VM ke BACKUP-NAS, ditambah `pg_dump` terjadwal ke NAS. VM baru dimasukkan ke jadwal uji restore.

9.3 Hal yang perlu dibereskan di Proxmox

- Disk root host terpakai 91 persen (85,62 dari 93,93 GiB). Harus diperiksa dan dikurangi karena root yang penuh dapat mengganggu layanan host dan backup.
- Ada peringatan repositori yang tidak siap produksi sedang aktif. Sebaiknya diganti ke repositori no-subscription yang stabil.
- Versi 8.2.2 sudah tertinggal. Pembaruan direncanakan terpisah, setelah backup penuh, dan tidak dilakukan bersamaan dengan migrasi aplikasi.

10. Strategi transisi dari sistem lama

- Aplikasi lama tetap berjalan sampai penggantinya siap.
- Data pegawai dan struktur organisasi diimpor awal dari HRIS ke modul Platform. Selama modul SDM baru belum ada, HRIS tetap menjadi sumber kebenaran.
- **Sandi tidak dibawa.** Hash MD5 tidak dipindahkan, dan sandi plaintext HRIS tidak boleh dibawa. Akun baru dibuat dengan sandi sementara dan kewajiban ganti sandi saat login pertama.
- Data historis yang tidak boleh hilang: riwayat payroll, keterlacakan roll dan bal produksi, nomor surat, dan sertifikat. Payroll sebaiknya berjalan paralel beberapa siklus sebelum sistem lama dimatikan.
- **Hardening darurat** pada sistem lama berjalan paralel dengan pembangunan baru, karena beberapa celah dapat dieksploitasi hari ini: rotasi kredensial yang terpapar di berkas konfigurasi, menutup eksekusi SQL dari klien di SIM, mencabut hak superuser database dari konfigurasi klien HRIS, dan memblokir akses ke `error.log` di SI WOPPIT.

10.1 Catatan tambahan: hardening yang belum tercakup

Hasil pembacaan dokumen analisis menunjukkan beberapa temuan yang belum ada di daftar hardening darurat di atas dan layak ditambahkan:

- **SI WOPPIT**: eskalasi hak akses lewat `user_role_id` pada form profil (pengguna biasa dapat menjadi Administrator). Perbaikannya kecil: hapus `user_role_id` dari field yang boleh diubah dan ambil `id_user` dari sesi server.
- **E-Arsip**: unggahan berkas ke direktori publik dengan tipe `octet-stream` diizinkan. Eksekusi PHP di folder unggahan perlu dinonaktifkan.
- **SI ELANG**: sandi MD5 dengan banyak sandi bawaan yang mudah ditebak, dan endpoint sertifikat yang dapat diakses langsung tanpa pemeriksaan di server.
- **HRIS**: ditemukan berkas berekstensi `.want_to_cry` yang mengindikasikan pernah ada infeksi ransomware (WannaCry) pada mesin atau server lokal. Perlu diperiksa sebelum VM baru dihubungkan ke jaringan yang sama.

11. Risiko dan hal yang perlu diwaspadai

Risiko	Penanganan
Pengembang tunggal	Cakupan dijaga kecil per tahap; dokumentasi dan pengujian ditulis sejak awal agar mudah diteruskan pihak lain

Risiko	Penanganan
Risiko terbesar rebuild adalah data, bukan kode	Migrasi data diuji dengan salinan data nyata
Kode pondasi (paket `ptkp-platform-core`: migrasi, seeder, penomoran) belum pernah dijalankan ke PostgreSQL dan Laravel sungguhan	Pengujian pertama pada database kosong adalah langkah wajib sebelum melanjutkan
Dokumentasi lama dibuat dari tinjauan dokumen, bukan eksekusi kode	Verifikasi klaim penting langsung ke kode, misalnya nama database HRIS yang berbeda-beda di dokumen (`db_ptkp`, `sygma_hris_ok`, `sygma_091014`)
Satu database berarti satu titik kegagalan	Backup berlapis dan uji restore rutin
Pemetaan identitas lama tidak seragam	NIP ada di HRIS, E-Arsip, SI WOPPIT, dan SI ELANG, tetapi tabel pengguna SIM tidak memiliki NIP, sehingga `legacy_identities` untuk SIM kemungkinan butuh pencocokan manual

12. Pertanyaan yang masih terbuka

Sudah terjawab

- **Aplikasi lama berdampingan:** tidak perlu menyediakan login terpusat untuk aplikasi lama. Ekosistem baru dibangun terpisah, aplikasi lama tetap berjalan dan diganti setelah penggantinya siap.
- **Sumber kebenaran absensi selama transisi:** HRIS. Absensi di SIM tetap berjalan sendiri sampai modul SIM dibangun ulang.
- **WOPPIT dan CMMS:** tidak digabung. WOPPIT (work order perbaikan IT) menjadi modul Layanan, sedangkan CMMS (pemeliharaan mesin) tetap menjadi bagian modul Manufaktur dan Mutu bersama SIM NEO.
- **Saluran notifikasi:** email dan WhatsApp. Email langsung memakai channel Laravel; WhatsApp membutuhkan gateway, sehingga pada tahap awal cukup disiapkan abstraksi channel-nya.

Masih terbuka

- **Format data HRIS** (kolom `emp_mstr` dan tabel organisasi) untuk skrip impor. Ditunda bersama skrip impor; sementara memakai data dummy lewat seeder.

- **Aturan scope hak akses E-Arsip** yang sebenarnya. Ditunda; fokus saat ini adalah membangun modul Platform.
- **Ukuran final VM produksi dan kebijakan retensi backup.** Ditunda.
- **Apakah SIM NEO dipisah menjadi deployment sendiri** yang memakai Platform lewat API dan SSO. Ditunda sampai SIM NEO dikerjakan (lihat bagian 14).

13. Status dan langkah berikutnya

13.1 Rencana awal

1. Rapikan disk dan repositori Proxmox.
2. Siapkan VM pengembangan dan jalankan migrasi Platform pada database kosong.
3. Tulis skrip impor pegawai dan organisasi dari HRIS.
4. Bangun login, antarmuka admin sederhana untuk organisasi, pegawai, dan hak akses.
5. Bangun modul E-Arsip di atas Platform.

13.2 Keputusan terbaru

Langkah 1 (rapikan disk Proxmox), langkah 2 (VM pengembangan), dan langkah 3 (skrip impor HRIS) **ditunda**. Pengembangan dimulai langsung di lingkungan lokal:

1. Jalankan migrasi Platform pada PostgreSQL lokal yang kosong dan perbaiki galat bila ada.
2. Isi data dummy lewat seeder (unit organisasi bertingkat, beberapa pegawai, beberapa peran) sebagai pengganti impor HRIS.
3. Tulis pengujian otomatis untuk layanan penomoran (termasuk permintaan bersamaan) dan pemeriksaan scope hak akses.
4. Bangun login dan antarmuka admin sederhana.
5. Bangun modul E-Arsip.

Langkah yang ditunda tetap diperlukan sebelum sistem baru dipakai sungguhan, terutama VM produksi, backup, dan impor data pegawai.

14. Standar jangka panjang (5 sampai 10 tahun)

Sistem ini dirancang agar tetap aman dan dapat dirawat dalam jangka 5 sampai 10 tahun. Keputusan inti: satu aplikasi modular dengan layanan Platform di balik interface, sehingga bagian mana pun dapat dipisah bila kebutuhan berubah.

Aspek	Standar
Arsitektur	Satu aplikasi Laravel, modul per domain, Platform sebagai inti. Layanan Platform (Directory, Numbering, Files, Audit) dipanggil lewat interface agar bisa dijadikan API HTTP tanpa mengubah pemanggil
Frontend	Inertia dan Vue. API (Sanctum) ditambahkan hanya saat ada kebutuhan nyata: mobile, fingerprint, atau pihak ketiga
Database	Satu PostgreSQL, schema per modul, akun database terpisah per modul, migrasi sebagai satu-satunya cara mengubah skema
Otorisasi	Deny-by-default, izin `modul.sumber.aksi` dengan scope, semua diperiksa di server; setiap policy memiliki tes otomatis
Keamanan akun	Hash Argon2id atau bcrypt, 2FA wajib untuk admin dan peran payroll, pembatasan percobaan login, sesi dicabut saat peran berubah
Keamanan data	Kolom sensitif (KTP, rekening, gaji) dienkripsi di level kolom, berkas di luar web root dan diunduh lewat controller berizin, audit log append-only
Keamanan infrastruktur	VM di segmen jaringan sendiri, database tidak dibuka ke jaringan umum, rahasia di luar repositori, `composer audit` dan `npm audit` rutin
Kemudahan dirawat	Teknologi yang umum dan sedikit paket pihak ketiga, tes otomatis termasuk tes arsitektur, catatan keputusan arsitektur (ADR), konvensi tertulis
Operasional	Backup terenkripsi dua lapis, uji restore terjadwal, pemantauan dan health check, runbook untuk kejadian umum
Data jangka panjang	Kebijakan retensi dan pengarsipan per jenis data, soft delete untuk dokumen resmi, abstraksi penyimpanan berkas agar bisa pindah ke storage lain

Aspek	Standar
Skala	Sangat lega untuk sekitar 300 pengguna; jalur pertumbuhan: indeks, partisi, replika baca, lalu pisah schema

Perawatan berkelanjutan

Sistem tanpa fitur baru tetap membutuhkan perawatan. Tanpa pembaruan keamanan, sistem ini menjadi legacy baru dalam 3 sampai 4 tahun. Karena itu diperlukan pemilik tetap dan jadwal perawatan tahunan:

- Laravel merilis versi mayor setiap tahun dan tiap versi mendapat dukungan keamanan terbatas, sehingga upgrade framework dilakukan kira-kira setiap 1 sampai 2 tahun.
- PHP dan PostgreSQL juga memiliki siklus dukungan terbatas. Masa dukung versi harus dicek di halaman resmi saat pengembangan dimulai, karena dapat berubah.
- Semakin sedikit paket tambahan dan semakin dekat ke fitur bawaan Laravel, semakin murah upgrade-nya.

Kapan SIM NEO dipisah

Keputusan ditunda sampai SIM NEO dikerjakan (modul terakhir). Bila saat itu ada tuntutan isolasi produksi, SIM dijadikan deployment tersendiri yang memakai Platform lewat API dan SSO. Karena batas modul dan schema `mes` sudah terpisah sejak awal, hal itu tidak membutuhkan penulisan ulang. Modul SDM atau payroll dipertimbangkan terpisah hanya bila ada kebutuhan isolasi data sensitif dari auditor atau kebijakan perusahaan.

Lampiran A. Ringkasan lima aplikasi lama

A.1 PTKP SIM NEO

- Rantai produksi: Stock Preparation, Mesin Kertas, Lab/QC, Finishing, Sortier, Pakzaal, Ekspedisi, dengan pelacakan nomor seri roll dan bal.
- Tata kelola mutu: SMT/ISO, dokumen induk, manajemen risiko, tiket PTKP/TKK (siklus Draft, Dalam Proses, Verifikasi SMT, Efektif atau Tidak Efektif, Ditutup).
- Database `simdb` (MySQL); koneksi baca ke ERP dan HRIS (PostgreSQL).
- Temuan utama: eksekusi SQL dari klien (kritis), kredensial hardcoded, MD5 tanpa salt, tanpa prepared statements, tabel dan versi skema yang tidak terkelola.

A.2 INSYSPRO HRIS

- Aplikasi desktop 2-tier yang berbicara langsung ke PostgreSQL. Empat grup: Admin, Absensi, Payroll, Absensi read-only.
- Alur absensi: mesin fingerprint, daemon pengambil log, tabel staging, mesin kalkulasi (keterlambatan, jam kerja netto, lembur), rekap harian.
- Alur payroll: periode gaji, karyawan aktif, komponen pendapatan dan potongan (BPJS, kasbon, koperasi, PPh 21), take home pay, slip dan rekap transfer bank.
- Di luar cakupan lama: portal pegawai dan pengajuan cuti mandiri.
- Temuan utama: kredensial superuser terbuka, sandi plaintext, jejak ransomware, denormalisasi tabel gaji, driver COM 32-bit.

A.3 E-Arsip

- Tiga peran (admin, trainer, peserta), 13 tabel surat dengan skema seragam.
- Format nomor surat: `[No Urut]/[Kode Dokumen]/PTKP/[Bulan Romawi]/[Tahun]`, contoh `01/ST/PTKP/X/2026`.
- Di luar cakupan lama: tanda tangan digital, disposisi berjenjang, OCR.
- Temuan utama: kolom sisa sistem ujian, penyimpanan ganda nota dinas, `sent_trainer_id` dipisah koma, unggahan berkas berisiko, IDOR, nomor ganda.

A.4 SI ELANG

- Alur belajar berurutan: Pre-Test, Materi PDF, Post-Test, Kesan (evaluasi), E-Sertifikat. Ada penguncian rute saat ujian.
- Tiga peran: admin, pengajar, siswa. Struktur departemen berupa pohon `el_kelas`.
- Temuan utama: MD5 tanpa salt, validasi alur belajar di `localStorage`, kunci sesi ganda yang rawan salah kunci, empat modul penilaian yang kembar, MyISAM tanpa foreign key, state ujian di tabel SQL.

A.5 SI WOPPIT

- Tiket perbaikan IT dengan tiga status: Menunggu Antrian, Proses, Selesai. Empat hasil pemeriksaan: dapat diperbaiki, perbaikan pihak ketiga, penggantian, tidak dapat diperbaiki. Mengacu pada formulir IT-FR-01/REV00.
- Enam tabel inti. Tiga peran: Administrator, Teknisi IT, User.
- Temuan utama: eskalasi hak akses lewat `user_role_id` (kritis), `id_user` dari form, `error.log` terbuka, nomor antrean dengan `MAX()+1`, tiket semua departemen terlihat oleh semua pengguna.

Lampiran B. Istilah

- **IDOR**: kelemahan ketika pengguna dapat mengakses atau mengubah data orang lain dengan mengganti ID pada permintaan.
- **RBAC**: kontrol akses berbasis peran.
- **Scope**: batas cakupan sebuah izin (milik sendiri, unit, unit beserta turunannya, atau semua).
- **Modular monolith**: satu aplikasi yang dibagi menjadi modul dengan batas tegas, tanpa dipecah menjadi layanan yang berjalan terpisah.
- **CAPA / PTKP / TKK**: permintaan tindakan koreksi dan pencegahan pada sistem manajemen mutu.
- **CMMS**: sistem manajemen pemeliharaan mesin.
- **ltree**: tipe data PostgreSQL untuk menyimpan dan mencari pohon hierarki.